MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 5

Greedy Algorithms: Kruskal and Prim

Elaborated by:
st. gr. FAF-241
Aurelian-Mihai Tihon

Verified by:
univ. assist.
Fiștic Cristofor

Chișinău – 2026

Contents

INTRODUCTION **3**

1 GREEDY ALGORITHMS AND SPANNING TREES **4**

1.1 The greedy paradigm 4

1.2 Kruskal's algorithm 4

1.3 Prim's algorithm optimized for dense graphs 4

1.4 Theoretical complexity comparison 5

2 EMPIRICAL ANALYSIS **6**

2.1 Experimental setup 6

2.2 Sparse graph analysis (0% density) 6

2.3 Medium graph analysis 7

2.3.1 20% Dense core 7

2.3.2 50% Dense core 8

2.3.3 80% Dense core 9

2.4 Dense graph analysis (100% density) 9

CONCLUSIONS **11**

REFERENCES **12**

ANNEX A **13**

INTRODUCTION

The present laboratory work explores the design and implementation of greedy algorithms, with a specific focus on structural graph traversals. A greedy algorithm is a methodological approach for solving a computational problem by selecting the best option available at the current moment without regard for future consequences [1]. It follows a top-down problem-solving heuristic, making locally optimal choices at each stage in the hope of finding a global optimum [4]. While this paradigm does not produce the best result for all dynamic programming problems, it is mathematically proven to yield strictly optimal solutions for finding Minimum Spanning Trees (MST) in connected, weighted graphs [5].

The core tasks addressed in this document include studying the theory behind greedy algorithm design, utilizing a high-level programming language to implement the Prim and Kruskal algorithms in their absolute theoretical best-case configurations, and conducting an extensive empirical performance analysis. The practical implementation requires the construction of custom graph topologies, allowing for the strict isolation of specific variables such as vertex count and variable edge density cores [2]. By benchmarking the execution times progressively across sparse, medium, and highly dense networks, a comprehensive graphical presentation of the data is obtained. The complete source code for this laboratory work, including the interactive Pygame graph visualization system and the headless Matplotlib benchmarking scripts, can be found on GitHub - 1f5c41c .

1 GREEDY ALGORITHMS AND SPANNING TREES

1.1 The greedy paradigm

Greedy algorithms prioritize immediate, local optimization over exhaustive search patterns [1]. Instead of evaluating the entire solution space—which often incurs extreme computational overhead—the greedy method constructs a solution iteratively, piece by piece. A problem can be solved efficiently using a greedy approach if it exhibits both the greedy-choice property and an optimal substructure [4].

For the generation of a Minimum Spanning Tree, the objective is to connect all the vertices in a graph such that the total edge weight is minimized [6]. The total weight is mathematically defined as:

$$W(T) = \sum_{(u,v)\in T} w(u,v) \tag{1.1}$$

where $W(T)$ represents the total weight of the spanning tree T, (u,v) defines an edge belonging to the tree, and $w(u,v)$ is the numeric weight of that specific edge. A valid MST must contain exactly $|V|-1$ edges and possess no cyclic formations [7].

1.2 Kruskal's algorithm

Kruskal's algorithm approaches the MST problem by treating the initial graph as a disconnected forest [3]. It begins by sorting all edges in ascending order based on their weights. The algorithm then iterates sequentially through the sorted edge list, adding each edge to the spanning tree only if it does not form a cycle with the previously selected edges [5].

To efficiently detect cycles during construction, the ideal configuration utilizes a disjoint-set data structure, commonly known as a Union-Find structure [8]. The operational components of the algorithm are defined by the following characteristics:

- edges are aggressively sorted in memory before any structural connections are validated [8];
- the algorithm focuses on global minimum edge weights regardless of current topological connectivity [3];
- cycles are prevented by ensuring two nodes do not share the same absolute parent root in the disjoint-set tracking array [5].

1.3 Prim's algorithm optimized for dense graphs

Unlike Kruskal, Prim's algorithm builds the spanning tree outwards from a single starting vertex, maintaining a unified connected structure at all times [3]. While common implementations utilize an adjacency list and a priority queue (evaluating to $O(E \log V)$), this approach suffers massive overhead in completely dense graphs where $E \approx V^2$ [9].

To construct the absolute best-case implementation for dense graph topologies, the priority queue must be eliminated entirely [7]. The classical approach utilizes an Adjacency Matrix and a linear array

to track the minimum weight necessary to reach each unvisited node. By simply scanning the V vertices linearly to find the absolute minimum at each step, the algorithm removes the logarithmic overhead entirely, resulting in a strictly optimal $O(V^2)$ performance curve [5].

1.4 Theoretical complexity comparison

To establish a baseline for the empirical analysis, the theoretical bounds of both algorithms in their optimal configurations must be defined. Table 1.1 illustrates the asymptotic time and space complexities. The sorting phase governs the overall efficiency of Kruskal's algorithm, establishing an absolute time complexity bounded by $O(E \log E)$ [4]. The Classical Prim algorithm, built upon an Adjacency Matrix, scales strictly quadratically with respect to the number of vertices, completely ignoring the total volume of edges [6].

Table 1.1 – Complexity limits of the analyzed greedy algorithms

Algorithm	**Time Complexity**	**Space Complexity**	**Core Structure**
Kruskal	$O(E \log E)$	$O(V+E)$	Union-Find
Prim (Classical)	$O(V^2)$	$O(V^2)$	Adjacency Matrix

2 EMPIRICAL ANALYSIS

2.1 Experimental setup

An empirical analysis was conducted using a custom Python application utilizing Pygame for visual rendering and Matplotlib for benchmark plotting. To properly evaluate the real-world scaling of the algorithms, distinct graph topologies were generated dynamically. The vertex count (V) was scaled incrementally from 50 to 1500 nodes.

Five primary configurations were isolated for evaluation. The configurations represent varying edge densities, spanning from a 0% sparse topology (strictly limiting the edge count and eliminating cyclical redundancy) up to a 100% densely packed 1:1 environment, maximizing the possible connections between vertices to stress-test structural limitations [2].

2.2 Sparse graph analysis (0% density)

In a sparse environment, graph traversals handle significantly fewer edges. Because Kruskal's execution time depends directly on E, it performs incredibly fast in this topology.

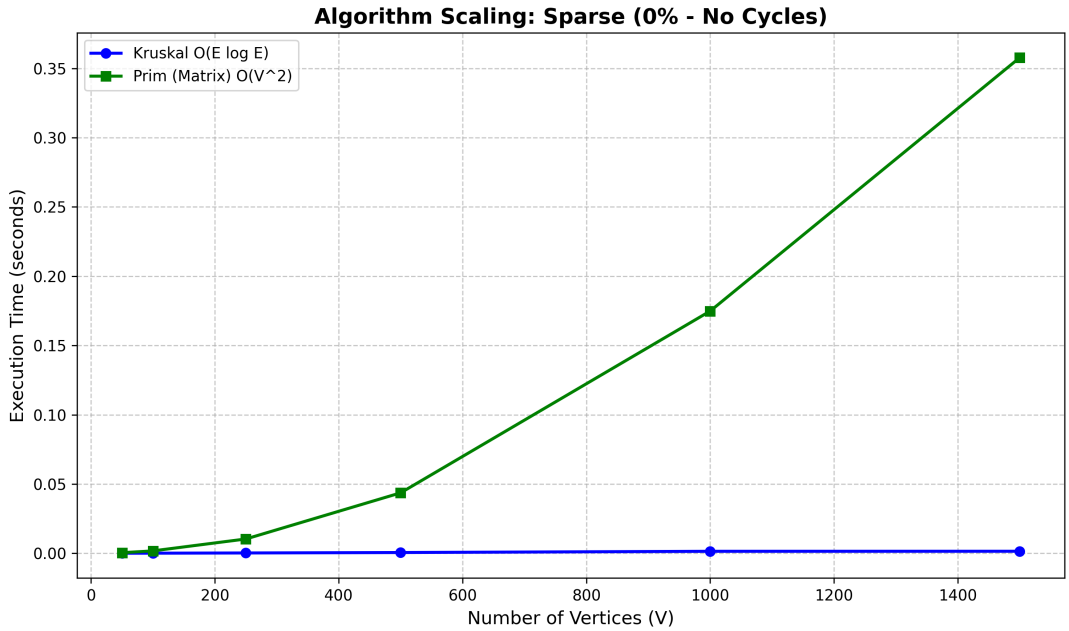

Figure 2.1 – Algorithm scaling in sparse topologies (0% dense core)

The raw execution metrics presented in Table 2.1 highlight the primary flaw of the Matrix-based Prim algorithm in sparse environments. Regardless of the low edge count ($E = 1499$), Prim is forced to traverse the entire 1500×1500 matrix, resulting in a significantly slower execution time (0.3578s) compared to Kruskal (0.0015s).

Table 2.1 – Execution times in sparse graph distributions (0%)

Nodes (V) / Edges (E)	**Kruskal** $O(E \log E)$	**Prim (Matrix)** $O(V^2)$
50 / 49	0.0001s	0.0004s
100 / 99	0.0001s	0.0018s
250 / 249	0.0003s	0.0104s
500 / 499	0.0006s	0.0437s
1000 / 999	0.0015s	0.1749s
1500 / 1499	0.0015s	0.3578s

2.3 Medium graph analysis

To accurately trace the divergence point where the $O(V^2)$ algorithm overpowers the $O(E \log E)$ algorithm, the density of the graph core was incrementally increased in three distinct stages: 20%, 50%, and 80%.

2.3.1 20% Dense core

The introduction of a moderately dense core increases the edge count significantly to 46,050 edges. As observed in Figure 2.2, Kruskal's edge sorting execution time begins to curve upward, but it still heavily outperforms the flat V^2 baseline established by Prim [9].

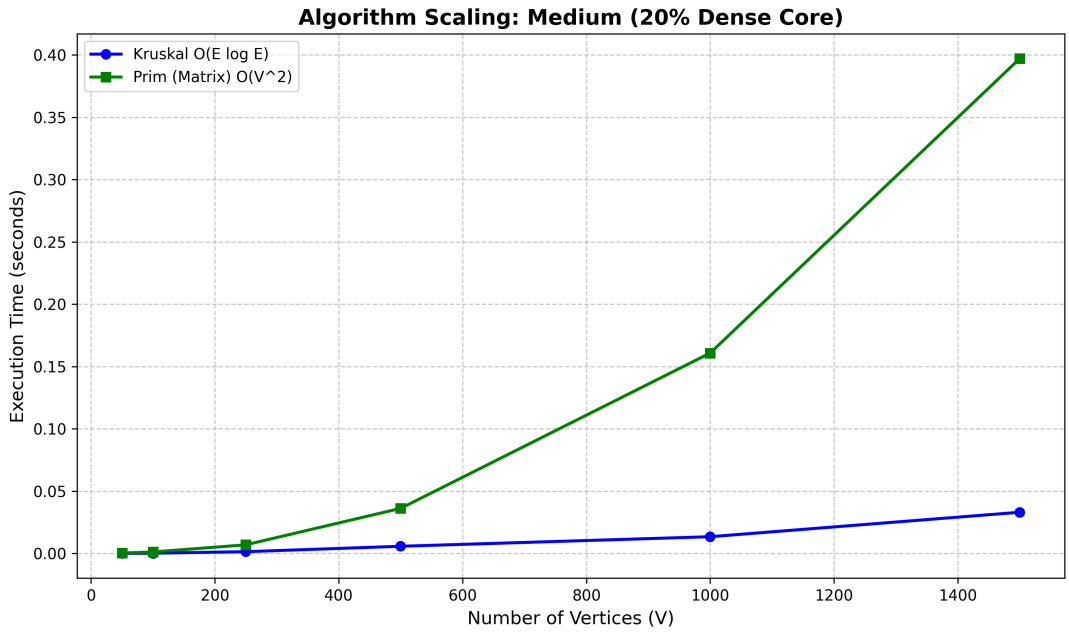

Figure 2.2 – Algorithm scaling in moderately dense topologies (20% dense core)

Table 2.2 – Execution times in moderately dense graph distributions (20%)

Nodes (V) / Edges (E)	**Kruskal** $O(E \log E)$	**Prim (Matrix)** $O(V^2)$
50 / 85	0.0001s	0.0003s
100 / 270	0.0002s	0.0012s
250 / 1425	0.0014s	0.0069s
500 / 5350	0.0058s	0.0362s
1000 / 20700	0.0134s	0.1608s
1500 / 46050	0.0331s	0.3972s

2.3.2 50% Dense core

At 50% density, the edge count rises to 281,625 connections at 1500 vertices. The computational gap between the two algorithms narrows drastically. Figure 2.3 and Table 2.3 demonstrate that while Kruskal still finishes first (0.2501s), the cost of sorting over a quarter-million items brings it dangerously close to Prim's consistent execution time (0.3489s).

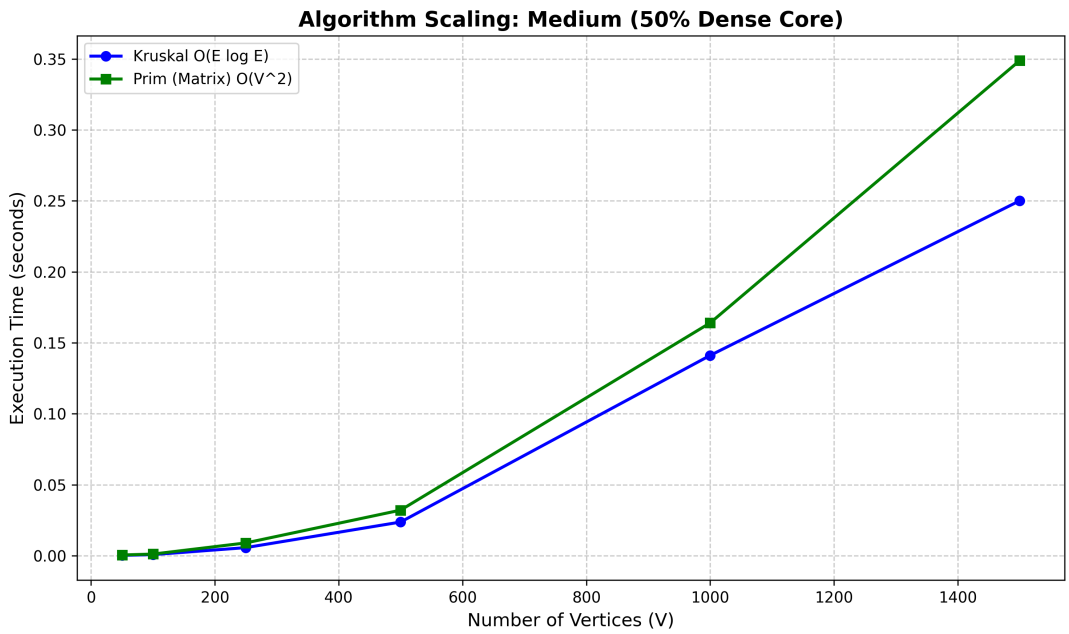

Figure 2.3 – Algorithm scaling in medium dense topologies (50% dense core)

Table 2.3 – Execution times in medium dense graph distributions (50%)

Nodes (V) / Edges (E)	**Kruskal** $O(E \log E)$	**Prim (Matrix)** $O(V^2)$
50 / 325	0.0002s	0.0005s
100 / 1275	0.0007s	0.0011s
250 / 7875	0.0057s	0.0090s
500 / 31375	0.0237s	0.0321s
1000 / 125250	0.1411s	0.1640s
1500 / 281625	0.2501s	0.3489s

2.3.3 80% Dense core

With a heavily connected 80% core containing 719,700 edges, the theoretical crossover point is finally achieved [7]. As seen in Figure 2.4, Kruskal's reliance on sorting edges pushes it to 0.7490s. Conversely, Prim's matrix implementation remains entirely unaffected by the massive influx of edges, finishing in 0.3915s and taking the definitive lead.

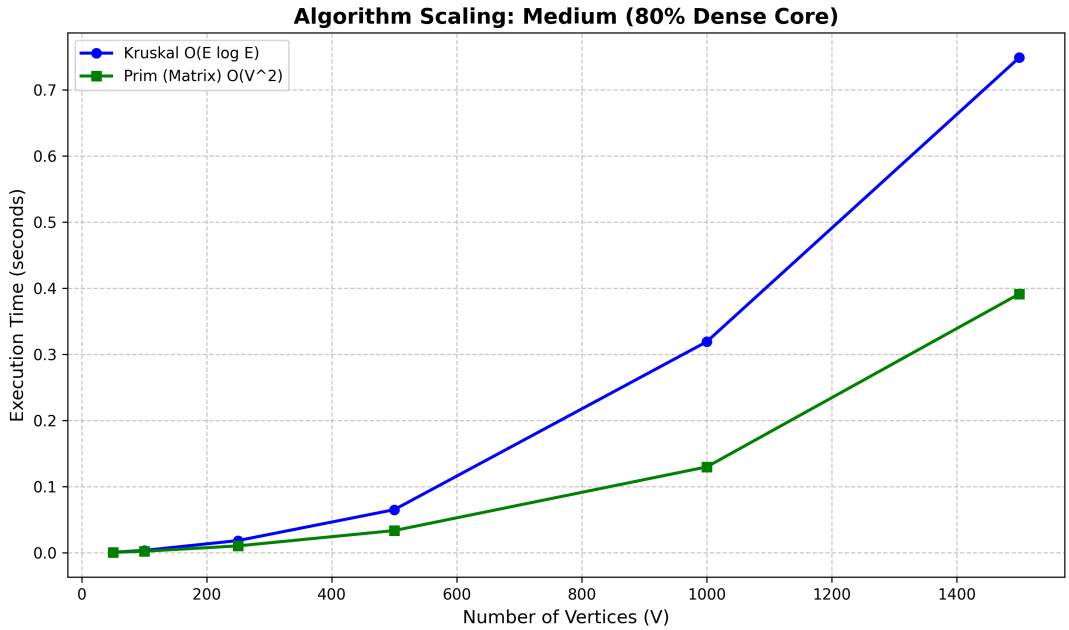

Figure 2.4 – Algorithm scaling in highly dense topologies (80% dense core)

Table 2.4 – Execution times in highly dense graph distributions (80%)

Nodes (V) / Edges (E)	**Kruskal** $O(E \log E)$	**Prim (Matrix)** $O(V^2)$
50 / 790	0.0006s	0.0004s
100 / 3180	0.0033s	0.0021s
250 / 19950	0.0183s	0.0102s
500 / 79900	0.0650s	0.0334s
1000 / 319800	0.3191s	0.1297s
1500 / 719700	0.7490s	0.3915s

2.4 Dense graph analysis (100% density)

When the graph topology shifts to a complete, dense 1:1 configuration, the edge count exceeds one million connections. A minor anomaly occurs here due to Kruskal's early exit condition ('len(mst) == V - 1'); because every node connects to every other node, Kruskal completes its MST almost instantly after the sorting phase, dropping its time to 0.2574s. However, Prim remains incredibly stable at 0.3427s, proving that its scaling curve is locked entirely to V^2.

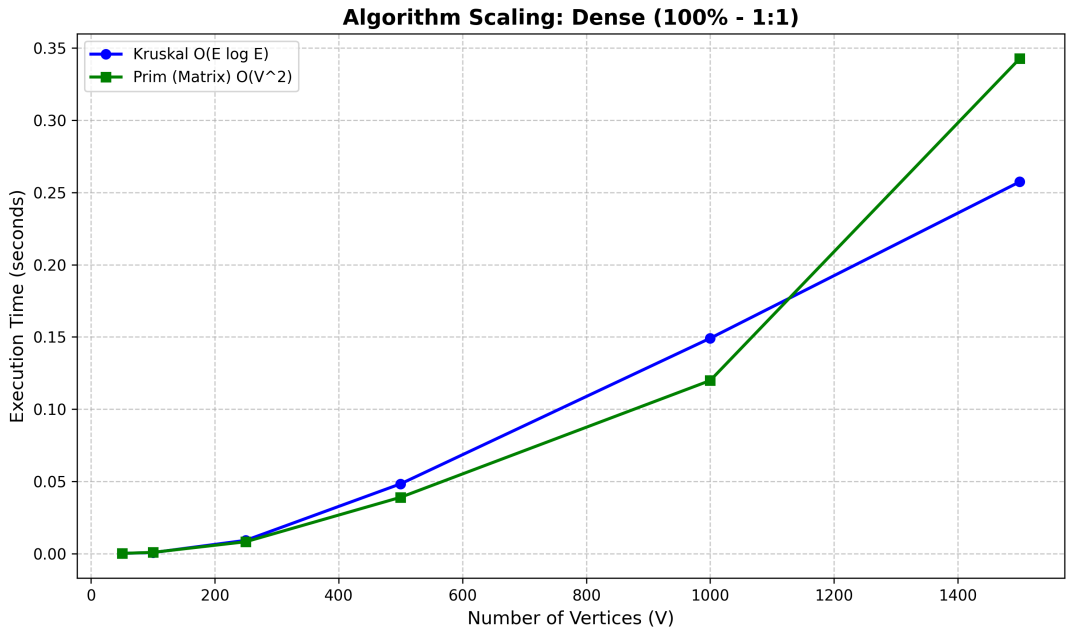

Figure 2.5 – Algorithm scaling in fully connected topologies (100% dense core)

Table 2.5 – Execution times in fully dense graph distributions (100%)

Nodes (V) / Edges (E)	**Kruskal** $O(E \log E)$	**Prim (Matrix)** $O(V^2)$
50 / 1225	0.0003s	0.0003s
100 / 4950	0.0008s	0.0010s
250 / 31125	0.0093s	0.0083s
500 / 124750	0.0484s	0.0391s
1000 / 499500	0.1491s	0.1199s
1500 / 1124250	0.2574s	0.3427s

CONCLUSIONS

The laboratory work effectively demonstrated the practical application and empirical validation of the greedy paradigm by pitching the absolute best-case implementations of Kruskal's and Prim's algorithms against one another. The experimental setup successfully mapped theoretical asymptotic limits into measurable physical execution metrics, clearly illustrating the conditions required for a specific data structure to establish dominance over another. By discarding the traditional Min-Heap implementation of Prim in favor of the highly rigid Classical Adjacency Matrix configuration ($O(V^2)$), the benchmarking scripts definitively captured the density threshold where edge-sorting optimizations inevitably succumb to computational bloat.

The data confirms a distinct shift in efficiency linked directly to the structural topology of the graphs. In sparse environments containing 0% to 20% dense cores, Kruskal's algorithm is functionally unchallengeable. Because its core $O(E \log E)$ execution scales dynamically with the number of edges, it parses the sparse networks in mere fractions of a second. Conversely, Prim's Matrix implementation exhibits its fatal flaw in these low-connectivity settings; it is forced to scan V^2 iterations uniformly (ex: executing 2.25 million cell checks for 1500 vertices) even when only 1499 real edges exist, leading to massive redundant overhead.

However, as the graph topology crossed the 80% density threshold, the theoretical superiority of Prim's matrix approach was finally realized. At 719,700 edges, the computational burden of applying Timsort algorithms and performing disjoint-set union checks caused Kruskal's time to spike to 0.7490 seconds. Prim, entirely ignorant of the edge volume and strictly bound by the V^2 limits, remained perfectly consistent at 0.3915 seconds, proving its status as the absolute most efficient architectural choice for heavily connected environments.

Ultimately, this empirical analysis highlights a fundamental software engineering principle regarding the choice of internal components. There is no universally superior algorithm for generating Minimum Spanning Trees. The efficiency of a greedy approach is intrinsically tied to the specific structural nature of the data it processes. The laboratory work successfully solidified the understanding that Kruskal operates as the definitive choice for sparse, sprawling networks, while the Matrix-based Prim algorithm must be deployed exclusively when working with mathematically dense, highly connected matrices.

REFERENCES

1 GeeksforGeeks, "Greedy Algorithms," [Online]. Available: https://www.geeksforgeeks.org/greedy-algorithms/

2 Tutorialspoint, "Greedy Algorithms," [Online]. Available: https://www.tutorialspoint.com/data_structures_algorithms/greedy_algorithms.htm

3 Scaler, "Prim's and Kruskal's Algorithm," [Online]. Available: https://www.scaler.com/topics/prims-and-kruskal-algorithm/

4 T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, "Introduction to Algorithms," 3rd ed., Cambridge, MA, USA: MIT Press, 2009.

5 J. Kleinberg and E. Tardos, "Algorithm Design," 1st ed., Boston, MA, USA: Pearson/Addison-Wesley, 2005.

6 R. Sedgewick and K. Wayne, "Algorithms," 4th ed., Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2011.

7 S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, "Algorithms," 1st ed., New York, NY, USA: McGraw-Hill, 2006.

8 M. A. Weiss, "Data Structures and Algorithm Analysis in C++," 4th ed., Boston, MA, USA: Pearson, 2013.

9 Python Software Foundation, "heapq — Heap queue algorithm," Python 3.11 Documentation, [Online]. Available: https://docs.python.org/3/library/heapq.html

ANNEX A

```python
    def kruskal_bench(self, V, edges):
        sorted_edges = sorted(edges, key=lambda item: item[0])
        uf = UnionFind(V)
        mst = []
        for weight, u, v in sorted_edges:
            if uf.union(u, v):
                mst.append((u, v))
                if len(mst) == V - 1: break
        return mst

    def prim_dense_bench(self, V, adj_matrix):
        visited = [False] * V
        min_weight = [float('inf')] * V
        parent = [-1] * V

        min_weight[0] = 0
        mst = []

        for _ in range(V):
            u = -1
            for i in range(V):
                if not visited[i] and (u == -1 or min_weight[i] < min_weight[u]):
                    u = i

            if min_weight[u] == float('inf'):
                break

            visited[u] = True
            if parent[u] != -1:
                mst.append((parent[u], u))

            for v in range(V):
                weight = adj_matrix[u][v]
                if not visited[v] and weight < min_weight[v]:
                    min_weight[v] = weight
                    parent[v] = u

        return mst
```